Core Java

Complete Notes

From basics to multithreading, collections, and Java 8

Personal Study Notes

Table of Contents

29. Date & Time API

What is Java?

Java is a high-level, object-oriented, platform-independent programming language.

Features

- Object-oriented
- Platform Independent (Write Once, Run Anywhere)
- Secure
- Robust
- Multithreaded
- Portable

Java Program Structure

```
public class Main {
        public static void main(String[] args) {
        System.out.println("Hello World");
        }
}
```

Explanation

- public → Accessible from anywhere
- class Main → defines a class named Main
- public static void main(String[] args) → the entry point of every Java application
- public → accessible by the JVM
- static → can be called without creating an object
- void → returns nothing
- main → the starting method
- String[] args → stores command line arguments

JVM, JRE and JDK

Note: This is one of the most frequently asked interview questions.

JDK (Java Development Kit)

Used to develop Java applications. Contains:

- JRE
- Compiler (javac)
- Debugging tools

JRE (Java Runtime Environment)

Used to run Java applications. Contains:

- JVM
- Libraries

JVM (Java Virtual Machine)

Responsible for:

- Running bytecode
- Memory management
- Garbage collection

Flow

```
Java Code (.java)
        ↓ javac
Bytecode (.class)
        ↓ JVM
Operating System
```

Variables

A variable stores data.

```
int age = 22;
```

Syntax: DataType variableName = value;

Datatypes tell Java the type of data we are going to store. There are two types of datatypes:

- Primitive Data Types
- Non-primitive Data Types

Primitive Data Types

Data Type	Size	Example
byte	1 byte	10
short	2 bytes	200
int	4 bytes	1000
long	8 bytes	100000L
float	4 bytes	10.5f
double	8 bytes	20.55
char	2 bytes	'A'
boolean	1 bit (logical)	true

Non-primitive Data Types

- String
- Array
- Class
- Interface

Operators

- Arithmetic: +, -, *, /, %
- Relational: >, <, >=, <=, ==, != (return true or false)
- Logical: &&, ||, !
- Bitwise Operators: &, |, ^, ~, <<, >>, >>>
- Assignment: =, +=, -=, *=, /=
- Ternary: ?:
- Unary Operator: ++, --, +, -, !

Input From User

```
import java.util.Scanner;

public class Test {
        public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter your name");
        String name = sc.nextLine();
        System.out.println(name);
        }
}
```

Type Casting

It is the process of converting a variable from one data type to another, either automatically by the compiler (implicit casting) or manually by the programmer (explicit casting).

Implicit Casting (Widening)

```
int a = 10;
double b = a;
System.out.println(b);
// Output: 10.0
```

Explicit Casting (Narrowing)

```
double x = 25.75;
int y = (int) x;
System.out.println(y);
// Output: 25
```

Note: Data loss occurs during narrowing / explicit casting.

Control Statements

Control statements control the flow of program execution. Types:

- Decision Making
- Looping
- Jump Statements

If Statement

Executes a block of statements only when the condition is true, otherwise skips the statements.

If-Else

- Chooses between two alternatives
- Only one block of statements is executed
- Used when exactly two possible outcomes are possible

Else-If Ladder

- Checks multiple conditions
- Conditions are checked from top to bottom
- The first true condition executes
- Remaining conditions are skipped
- else is optional

Nested If

- An if inside another if
- Used when the second condition depends on the first

Switch Statement

- Chooses one block from multiple options based on an exact value
- Works with byte, short, char, int, String, and enum
- Case values must be constants
- default runs if no case matches
- break prevents fall-through

Difference: If vs Switch

if	switch
Can check ranges (>, <, >=)	Checks exact values only
Supports logical operators	Does not compare ranges
Better for complex conditions	Better for menu / value selection

For Loop

- Repeats code a known number of times
- for(initialization; condition; update) { }
- Initialization runs once
- Condition is checked before each iteration
- Update runs after each iteration

While Loop

- Repeats while a condition is true
- while(condition) { }
- Entry-controlled loop
- May execute zero times

Do-While Loop

- Executes at least once
- do { } while(condition);
- Exit-controlled loop
- Executes the body first, then checks the condition

Difference: for vs while vs do-while

for	while	do-while
Known iterations	Unknown iterations	Executes at least once
Entry-controlled	Entry-controlled	Exit-controlled

Break

Immediately exits the loop or switch.

```
for (int i = 1; i <= 10; i++) {
        if (i == 5)
        break;
        System.out.println(i);
}
// Output: 1 2 3 4
```

Continue

Skips the current iteration and continues with the next one.

```
for (int i = 1; i <= 5; i++) {
        if (i == 3)
        continue;
        System.out.println(i);
}
// Output: 1 2 4 5
```

Arrays

- An array is a non-primitive data type and a collection of elements of the same data type stored in a contiguous memory location
- Example: int[] marks = {12, 34, 55, 5};
- Stores multiple values in one variable of the same data type
- Easy to access using index
- Simplifies looping and data processing
- Size is fixed after creation
- Index starts from zero
- The last index is length - 1
- Arrays are objects in Java
- Stored in heap memory

Array Declaration

- Method 1: int[] arr;
- Method 2: int arr[];

Array Creation

```
int[] arr = new int[5];
```

Default Values

Data Type	Default Value
int	0
double	0.0
boolean	false
char	'\u0000'
String / Object	null

Array Initialization

```
// Method 1
int[] arr = {10, 20, 30, 40, 50};

// Method 2
int[] arr = new int[5];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
arr[3] = 40;
arr[4] = 50;
```

Accessing Elements

```
System.out.println(arr[0]);
// Array length is a property, not a method
arr.length;
```

Common Exceptions

ArrayIndexOutOfBoundsException:

```
int[] arr = {10, 20, 30};
System.out.println(arr[5]);
// Error: invalid index
```

NullPointerException:

```
int[] arr = null;
System.out.println(arr.length);
```

Strings

- A String is a non-primitive datatype and a sequence of characters
- Example: String name = "Nitish";
- Used to store text
- Most Java applications process text (names, emails, messages, etc.)
- String is a class in Java (java.lang.String)
- It is immutable (cannot be changed after creation)
- Stored in the String Constant Pool when created using string literals
- String overrides the equals() method to compare values

Ways to Create Strings

1. Using string literals:

```
String str = "Java";
```

- Stored in the String Constant Pool
- If the same value already exists, Java reuses the existing object

2. Using the new keyword:

```
String str = new String("Java");
```

- Creates a new object in heap memory
- Does not reuse the existing literal object

String Constant Pool (SCP)

```
String s1 = "java";
String s2 = "java";
// Only one object is created in the SCP
// Both s1 and s2 refer to the same object
```

Comparing Strings

Using == (checks memory / reference):

```
String s1 = "Java";
String s2 = "Java";
System.out.println(s1 == s2);   // true

String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1 == s2);   // false (different objects)
```

Using equals() (checks content / value):

```
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1.equals(s2));   // true
```

Comparison: == vs equals()

==	equals()
Compares references	Compares values
Can return false for same text	Returns true if contents are the same
Operator	Method

Note: Use equals() to compare string values (interview tip).

Common String Methods

- length() → finds the length of the string: str.length();

- charAt() → gets the character at a specific index: str.charAt(4);
- toUpperCase() → converts to capital letters: str.toUpperCase();
- toLowerCase() → converts to lower case letters: str.toLowerCase();
- substring() → extracts part of a string: str.substring(startIndex) or str.substring(startIndex, endIndex);
- replace() → replaces part of a string: str.replace(oldStr, newStr);
- trim() → removes leading and trailing spaces: str.trim();
- split() → breaks a string into an array of substrings

StringBuilder

Mutable String but not thread-safe.

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" programming");
System.out.println(sb);
// Output: Java programming
```

StringBuffer

Also mutable, but thread-safe.

String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	No	No	Yes
Performance	Slow for frequent modifications	Fast	Slower than StringBuilder

Q. Why are Strings Immutable?

Once a String object is created, its value cannot be changed, because instead of modifying the existing object, Java creates a new object.

Q. What is the String Constant Pool (SCP)?

The String Constant Pool is a special memory area inside the heap where Java stores string literals.

Q. Difference between concat() and append()

concat()	append()
Belongs to String	Belongs to StringBuilder / StringBuffer
Returns a new String	Modifies the existing object
String remains immutable	Mutable operation
Slower for repeated concatenation	Faster for repeated concatenation

Methods

A method is a block of code that performs a specific task; it is executed only when it is called.

Syntax:

```
accessModifier returnType methodName(parameters) {
        // body
}

public static void greet() {
        System.out.println("Hello");
}

// Calling the method by name
greet();
```

Advantages

- Code Reusability
- Better Readability
- Easier Maintenance
- Reduces Code Duplication

Parameters vs Arguments

Parameter: a variable in the method definition. Example: public static void add(int a, int b) — a and b are parameters.

Argument: the value passed while calling the method. Example: add(10, 20).

Types of Methods

There are 4 combinations:

```
// 1. No Parameters, No Return Value
public static void greet() {
        System.out.println("Hello");
}
// Call: greet();

// 2. Parameters, No Return Value
public static void greet(String name) {
        System.out.println(name);
}
// Call: greet("Nitish");

// 3. No Parameters, Return Value
public static int getNumber() {
        return 100;
}
// Call: int n = getNumber();

// 4. Parameters, Return Value
public static int add(int a, int b) {
        return a + b;
}
// Call: int sum = add(10, 20);
```

return Keyword

Used to send a value back to the caller. Example: return a + b; — once return executes, the method ends.

Call by Value

Java is pass-by-value.

```
public static void change(int x) {
        x = 100;
}

public static void main(String[] args) {
        int a = 10;
        change(a);
        System.out.println(a);
}
// Output: 10 — because Java passes a copy of the value
```

Method Overloading

Method Overloading means having multiple methods with the same name but different parameter lists in the same class. It is an example of compile-time polymorphism.

Rules

- Method name must be the same
- Parameters must be different

The difference can be in:

- Number of parameters
- Types of parameters
- Order of parameters

Note: Return type alone cannot overload a method.

Variable Scope

Variables have different scopes. There are three types:

A. Local Variable

Declared inside a method.

```
public static void test() {
        int a = 10;
}
```

- Exists only inside the method

- Cannot be accessed outside
- Has no default value

B. Instance Variable

Declared inside a class but outside methods.

```
class Student {
        int age;
}
```

- Each object gets its own copy
- Gets a default value

C. Static Variable

Declared using static.

```
class Student {
        static String clg = "LPU";
}
```

- Shared by all objects
- Only one copy exists

Recursion

A method calling itself is called recursion.

```
public class Demo {
        static void print(int n) {
        if (n == 0)
        return;
        System.out.println(n);
        print(n - 1);
        }

        public static void main(String[] args) {
        print(5);
        }
```

```
}
```

Note: Every recursive function must have a stopping condition, otherwise it causes a StackOverflowError.

Varargs (...)

Allows passing any number of arguments.

```
public static int sum(int... numbers) {
        int total = 0;
        for (int n : numbers) {
        total += n;
        }
        return total;
}

// Calls:
sum(10);
sum(10, 20);
sum(10, 20, 30);
```

Note: Varargs should be the last parameter. Example: display(String name, int... marks)

Passing an Array to a Method

```
public static void printArray(int[] arr) {
        for (int n : arr) {
        System.out.println(n);
        }
}

// Calling
int[] arr = {10, 20, 30};
printArray(arr);
```

Returning an Array

```
public static int[] createArray() {
        return new int[] {10, 20, 30};
}
// Calling: int[] arr = createArray();
```

OOP (Object Oriented Programming)

Q. What is OOP?

OOP (Object-Oriented Programming) is a programming concept that organizes programs using objects instead of only functions. An object contains:

- Data (fields / variables)
- Behavior (methods)

Why OOP?

Without OOP:

- Difficult to manage large applications
- Code duplication
- Hard to maintain

With OOP:

- Reusability
- Security
- Easy maintenance
- Better organization
- Real-world modeling

Real-World Example

Think of a car. Every car has:

Properties (state):

- Brand
- Color
- Price
- Model

Behaviors (actions):

- start()
- stop()
- accelerate()

What is a Class?

A class is a blueprint or template for creating an object. It defines:

- Variables (attributes)
- Methods (behavior)

```
class Student {
        String name;
        int age;

        void study() {
      System.out.println("Studying..");
        }
}
```

Here, variables: name, age. Methods: study().

What is an Object?

An object is an instance of a class. Memory is allocated only when an object is created.

```
Student s1 = new Student();
```

Breaking this line down:

- Student → class name
- s1 → reference variable
- new → allocates memory
- Student() → constructor

Accessing Variables and Methods

```
Student s1 = new Student();
s1.name = "Nitish";
s1.age = 22;
s1.study();
```

Note: We can create multiple objects of a class, and each object has its own copy of instance variables — each object stores its own data.

Instance Variables

Declared inside a class but outside methods.

```
class Student {
        String name;
        int age;
}
```

Instance Methods

Methods that belong to an object; an object is needed to call them.

Object Reference

```
Student s1 = new Student();
```

s1 is not the object — it stores the reference (address) of an object.

Constructors

A constructor is a special method that is automatically called when an object is created. Its main purpose is to initialize the object's data.

```
Student s = new Student();
// The constructor Student() is called automatically.
```

Why Do We Need a Constructor?

Without a constructor:

```
Student s = new Student();
s.name = "Nitish";
s.age = 22;
```

With a constructor:

```
Student s = new Student("Nitish", 22);
// The object is initialized immediately.
```

Rules of Constructors

- The constructor name must be the same as the class name
- Constructors have no return type (not even void)
- Called automatically when an object is created
- Constructors can be overloaded
- A constructor cannot be inherited

Types of Constructors

- Default Constructor — automatically provided by the compiler if you don't write any constructor
- No-Argument Constructor — created by the programmer but has no arguments
- Parameterized Constructor — used to initialize objects with values

Constructor Overloading

Just like methods:

```
class Student {
        Student() { }
        Student(String name) { }
        Student(String name, int age) { }
}
```

Different parameter lists.

Constructor vs Method

Constructor	Method

Same name as class	Any valid name
No return type	Must have a return type
Called automatically	Called manually
Initializes object	Performs operations
Runs once per object creation	Can be called many times

Constructor Chaining (this())

One constructor can call another constructor in the same class.

```
class Student {
        Student() {
        this("nitish");
      System.out.println("Default");
        }
        Student(String name) {
        System.out.println(name);
        }
}
```

- this() must be the first statement in the constructor
- Used to avoid duplicate code

this Keyword

```
class Student {
        String name;
        Student(String name) {
        this.name = name;
        }
}
```

Without this, name = name; both refer to the parameter. With this, this.name refers to the instance variable, and name refers to the constructor parameter.

Five Important Uses of this

1. Refers to the current object's instance variable — to differentiate instance variables from local variables or parameters with the same name:

```
this.name = name;
```

2. Call the current class method:

```
class Student {
        void display() {
        this.show();
        }
        void show() {
      System.out.println("Hello");
        }
}
```

3. Call the current class constructor (this()):

```
class Student {
        Student() {
        this("Nitish");
      System.out.println("Default");
        }
        Student(String name) {
        System.out.println(name);
        }
}
```

- this() must be the first statement in the constructor
- Used for constructor chaining
- Avoids duplicate code

4. Pass the current object as an argument:

```
class Student {
        void display() {
        print(this);
        }
        void print(Student obj) {
        System.out.println("Object received");
        }
}
```

5. Return the current object:

```
class Student {
        Student getStudent() {
        return this;
        }
}
```

Used in method chaining and the Builder Pattern.

Difference between this and super

this	super
Refers to current class object	Refers to parent class object
Calls current constructor	Calls parent constructor
Access current variables	Access parent variables

Note: this cannot be used in a static method.

Four Pillars of OOP

1. Encapsulation

It is the process of wrapping data (variables) and methods into a single unit (class) and restricting direct access to the data.

In simple words: hide the data and provide controlled access through methods.

Real-Life Example

Think of an ATM machine. You can deposit money, withdraw money, and check balance — but you cannot directly access the bank's database. The ATM hides the internal implementation. This is Encapsulation.

Why Encapsulation?

```
class Student {
        String name;
        int age;
}

Student s = new Student();
s.age = -20;
```

Age cannot be negative, but Java allows it because the variable is public / default accessible. To protect the data:

```
private int age;
// Now no one can modify it directly.
```

How to Achieve Encapsulation

Step 1: Make variables private.

```
private String name;
```

Step 2: Provide getter and setter methods.

```
public String getName() {
        return name;
}

public void setName(String name) {
        this.name = name;
}
```

Getter → reads the data. Setter → modifies the data.

Data Validation

One of the biggest advantages of encapsulation:

```
public void setAge(int age) {
        if (age > 0) {
        this.age = age;
        }
}

student.setAge(-10);   // Nothing changes — invalid data is rejected.
```

Benefits of Encapsulation

- Data hiding
- Better security
- Controlled access
- Easier maintenance
- Flexible code
- Validate before updating data

Encapsulation vs Data Hiding

Encapsulation	Data Hiding
Wraps data and methods together	Restricts direct access to data
Achieved using classes	Achieved using the private access modifier
Broader concept	Part of encapsulation

Note: Encapsulation is the process of binding data and methods into a single class and providing controlled access to the data using methods.

2. Inheritance

Inheritance is the process by which one class acquires the properties (variables) and behaviours (methods) of another class. It promotes code reusability.

Real-Life Example

Think of: a car is a vehicle. A car automatically has common features of a vehicle like Engine, Wheels, start(), stop(). The Car class can also have its own features like Sunroof, Music System.

```
class Animal {
        void eat() {
        System.out.println("Animal is eating");
        }
}

class Dog extends Animal {
        void bark() {
        System.out.println("Dog is barking");
        }
}

public class Demo {
        public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
        }
}
// Output:
// Animal is eating
// Dog is barking
```

Here, Animal → Parent (superclass), Dog → Child (subclass).

Why Use Inheritance?

Without inheritance, the same code is repeated:

```
class Car {
        void start() { }
}

class Bike {
        void start() { }
}
```

With inheritance, both classes reuse the start() method:

```
class Vehicle {
        void start() { }
}

class Car extends Vehicle { }
class Bike extends Vehicle { }
```

Terminology

- Superclass → parent class
- Subclass → child class
- extends → keyword used for inheritance
- IS-A relationship → child is a type of parent

Types of Inheritance

- Single Inheritance — one parent → one child
- Multilevel Inheritance — Animal → Dog → Puppy
- Hierarchical Inheritance — Animal → Dog and Cat
- Multiple Inheritance (not supported with classes) — one child has multiple superclasses/parents
- Hybrid Inheritance — combination of multiple types; not supported with classes, possible using interfaces

Note: Java does not support multiple inheritance with classes because it can create ambiguity (the Diamond Problem).

super Keyword

super refers to the parent class object. There are 3 main uses:

- Access parent variable: super.color;
- Call parent method: super.sound();
- Call parent constructor: super();

Note: If we don't write super(), Java inserts it automatically.

Constructor Calling Order

```
class A {
        A() {
        System.out.println("A");
        }
}

class B extends A {
        B() {
        System.out.println("B");
        }
}

B obj = new B();
// Output: A  then  B
```

The parent constructor always executes first.

Method Overriding

If the child class provides its own implementation of a parent class method, it is called Method Overriding.

```
class Animal {
        void sound() {
        System.out.println("Animal sound");
        }
}

class Dog extends Animal {
        @Override
        void sound() {
        System.out.println("Dog Bark");
        }
}
```

Rules of Method Overriding

- Same method name
- Same parameters

- Same return type (or a covariant return type)
- Cannot reduce method visibility
- final methods cannot be overridden
- static methods are hidden, not overridden

@Override Annotation

```
@Override
void display() {
}
```

Benefits:

- Compiler checks if you are actually overriding
- Prevents accidental mistakes

Difference: Overloading vs Overriding

Overloading	Overriding
Same class	Parent & child
Different parameters	Same parameters
Compile-time polymorphism	Runtime polymorphism
Return type alone cannot overload	Return type must be compatible

final Keyword in Inheritance

- final Class → cannot be inherited
- final Method → cannot be overridden
- final Variable → cannot be changed

IS-A Relationship

IS-A means inheritance. If one object is a type of another object, then there is an IS-A relationship. Java uses the extends keyword, or implements for interfaces.

HAS-A Relationship

HAS-A means one class contains another class as a member (object). This is called Composition or Aggregation, depending on ownership.

A simple way to find out if it is IS-A or HAS-A: "Is a dog an animal?" → Yes → IS-A. "Does a laptop have a processor?" → Yes → HAS-A.

3. Polymorphism

The word polymorphism comes from two Greek words: Poly → many, Morph → forms. So polymorphism means one thing behaving in many different ways.

Real-Life Example

Think about a person: the same person can be a son at home, an employee at work, and a friend with friends — same person, different behaviour. This is polymorphism.

Types of Polymorphism

Java has 2 types:

- Compile-time polymorphism (Method Overloading)
- Runtime Polymorphism (Method Overriding)

1. Compile-Time Polymorphism

Also called Static Binding / Early Binding. Achieved by method overloading.

```
class Calculator {
        int add(int a, int b) {
        return a + b;
        }
        int add(int a, int b, int c) {
        return a + b + c;
        }
}
```

The compiler decides which method to call.

2. Runtime Polymorphism

Also called Dynamic Binding / Late Binding. Achieved by method overriding.

```
class Animal {
        void sound() {
        System.out.println("Animal sound");
        }
}

class Dog extends Animal {
        @Override
        void sound() {
        System.out.println("Dog barks");
        }
}

// In main:
Animal obj = new Dog();
obj.sound();
// Output: Dog barks
```

Notice: the reference is Animal, but the object is Dog. Java decides at runtime which method to execute.

Why Does It Work? — Upcasting

```
Animal obj = new Dog();
```

This is called Upcasting: creating a parent class reference for a child class object.

What Can We Access?

```
class Animal {
          void eat() {
       System.out.println("Eating..");
          }
}

class Dog extends Animal {
          void bark() {
       System.out.println("Barking..");
          }
}

// In main:
Animal obj = new Dog();
obj.eat();         // allowed
obj.bark();   // compile-time error — reference type is Animal
```

Dynamic Method Dispatch

```
class Animal {
        void sound() {
      System.out.println("Animal");
        }
}

class Dog extends Animal {
        @Override
        void sound() {
        System.out.println("Dog");
        }
}

class Cat extends Animal {
        @Override
        void sound() {
        System.out.println("Cat");
        }
}

// In main:
Animal a;
a = new Dog();
a.sound();
a = new Cat();
a.sound();
// Output: Dog  then  Cat
```

Same reference, different objects, different behaviour. This is Dynamic Method Dispatch.

Downcasting

```
Animal obj = new Dog();
Dog d = (Dog) obj;   // Downcasting
d.bark();   // works
```

Unsafe downcasting:

```
Animal obj = new Animal();
Dog d = (Dog) obj;
// Output: ClassCastException — the object is actually an Animal, not a Dog
```

Method Overloading vs Method Overriding

Method Overloading	Method Overriding
Same class	Parent & child
Different parameters	Same parameters
Compile-time	Runtime
Static Binding	Dynamic Binding
Example: add()	Example: sound()

4. Abstraction

Abstraction means hiding the implementation details and showing only the essential functionality.

In simple words: show "what" an object does, hide "how" it does it.

Real-Life Example (ATM)

We insert our ATM card and withdraw money. We know: withdraw money, check balance, deposit money. But do we know how the bank server processes the transaction internally? The internal implementation is hidden. This is called Abstraction.

How Is Abstraction Achieved in Java?

Java provides 2 ways:

- Abstract class
- Interface

What Is an Abstract Class?

An abstract class is a class declared using the abstract keyword.

```
abstract class Animal {
}
```

Important rules:

- An abstract class cannot be instantiated (an object cannot be created)
- An abstract class can have a constructor — it runs when a child object is created
- An abstract class can contain normal methods
- An abstract class can contain abstract methods

What Is an Abstract Method?

An abstract method has no body — only a declaration.

```
abstract void sound();
```

Note: No {} — no implementation. The child class must implement it.

```
abstract class Animal {
        abstract void sound();

        void eat() {
        System.out.println("Animal is eating");
        }
}

class Dog extends Animal {
        @Override
        void sound() {
        System.out.println("Dog Barks");
        }
}

public class Demo {
        public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
        d.eat();
        }
}
// Output:
// Dog barks
```

```
// Animal is eating
```

Why Do We Need Abstract Classes?

Suppose every animal has eat() and sleep(), but every animal makes a different sound: Dog → bark, Cat → Meow, Cow → Moo. Instead of writing an empty sound() inside Animal, we force every child to implement it: abstract void sound();

What Happens If a Child Does Not Implement the Abstract Method?

```
abstract class Animal {
        abstract void sound();
}

class Dog extends Animal {
}
// Compile-time error
```

The child class must either implement sound(), or be declared abstract itself.

Q&A: Abstract Class Rules

- Can an abstract class have variables? Yes.
- Can abstract classes have constructors? Yes.
- Can an abstract class have static methods? Yes.
- Can an abstract class have final methods? Yes.

```
abstract class Animal {
        Animal() {
        System.out.println("Animal Constructor");
        }
}

class Dog extends Animal {
        Dog() {
        System.out.println("Dog Constructor");
        }
}
// Output: Animal Constructor  then  Dog Constructor
```

Difference between Abstract Method and Normal Method

Abstract Method	Normal Method
No body	Has body
Ends with ;	Uses {}
Must be implemented by child	Can be inherited directly

Must Remember

- abstract is a keyword
- An abstract class cannot be instantiated
- It can contain abstract methods and normal methods
- A child class must implement all abstract methods unless it is also declared abstract
- Abstract classes can have constructors, variables, static methods, and final methods

Interface

An interface is a blueprint of a class. It contains behavior (method declarations) that implementing classes must provide.

Think of it as a contract — if you implement this interface, you must implement all its methods.

Real-Life Example

Imagine a USB port: a keyboard, mouse, and printer all connect to the same USB port. The computer doesn't care whether it's a keyboard or a printer — it only knows the device follows the USB standard. Similarly, Java says: this class follows this interface.

```
interface Animal {
        void sound();
}
```

Notice: interface is a keyword; no abstract keyword needed; the method has no body, only the declaration.

Implementing an Interface

Java uses the implements keyword.

```
interface Animal {
        void sound();
}

class Dog implements Animal {
        @Override
        public void sound() {
        System.out.println("Bark");
        }
}

// In main:
Dog d = new Dog();
d.sound();
// Output: Bark
```

Important Rules

- Cannot create an object of an interface
- A class implements an interface, not extends
- Interface methods are public abstract by default
- Variables inside an interface are public static final by default

```
interface Demo {
        int val = 10;
        // Treated as: public static final int val = 10;
}
```

Multiple Inheritance Using Interfaces

Java does not support multiple inheritance with classes, but it does allow multiple inheritance using interfaces.

```
interface Camera {
        void click();
}

interface MusicPlayer {
        void play();
}

class Smartphone implements Camera, MusicPlayer {
        @Override
        public void click() {
        System.out.println("Photo Clicked");
        }
        @Override
        public void play() {
        System.out.println("Playing Music");
        }
}
```

Interface vs Abstract Class

Abstract Class	Interface
Uses extends	Uses implements
Can have constructors	Cannot have constructors
Can have instance variables	Only constants (public static final)
Can have normal methods	Methods are abstract by default (plus default / static methods since Java 8)
Supports single inheritance	Supports multiple inheritance

When to Use an Abstract Class?

Use when classes share common code. Example: Employee — all employees have Name, Id, and displayDetails(); only salary calculation differs. Use abstract classes.

When to Use an Interface?

Use when different classes have different implementations but must provide the same capability. Example: UPI payment — Google Pay, PhonePe, and Paytm all implement pay() differently. Use an interface.

Default Method (Java 8)

Before Java 8, interfaces could not implement methods. Java 8 introduced default methods.

```
interface Animal {
        void sound();

        default void eat() {
    System.out.println("Eating");
        }
}
```

The implementing class may override eat(), but it does not have to.

Static Method (Java 8)

Interfaces can also have static methods.

```
interface Demo {
        static void display() {
    System.out.println("Hello");
        }
}

// Call it like this:
Demo.display();
```

Functional Interface

An interface with exactly one abstract method.

```
@FunctionalInterface
interface Calculator {
        int add(int a, int b);
}
```

Functional interfaces are used with lambda expressions.

static Keyword

A static member belongs to the class, not to any individual object.

Normally, each object has its own copy of an instance variable, but a static variable has only one copy, shared by all objects.

Why Do We Need Static?

Suppose every student belongs to the same college. Without static, every object stores "LPU" separately — if you create 10,000 students, "LPU" is stored 10,000 times. Instead, using static, only one copy of the college name exists.

Static Variable Example

```
class Student {
        String name;
        static String college = "LPU";

        Student(String name) {
        this.name = name;
        }

        void display() {
        System.out.println(name + " " + college);
        }
}

public class Demo {
        public static void main(String[] args) {
        Student s1 = new Student("Nitish");
        Student s2 = new Student("Mehul");
        s1.display();
        s2.display();
        }
}
// Output:
// Nitish LPU
// Mehul LPU
```

Both objects share the same college name. Changing a static variable — Student.college = "IIT"; — updates it for both objects, because there is only one shared copy.

Static Method

A static method belongs to the class.

```
class Demo {
        static void display() {
      System.out.println("Hello");
        }
}

// Call it like this:
Demo.display();   // No object required.
```

Can a Static Method Access Instance Variables?

Directly, no — it causes a compile-time error, because a static method belongs to the class but the instance variable belongs to an object; the compiler doesn't know which object's variable you mean.

It can access one by first creating an object:

```
class Demo {
        int x = 10;
        static void display() {
        Demo d = new Demo();
        System.out.println(d.x);
        }
}
```

Can Instance Methods Access Static Variables?

Yes — an object can always access class-level members.

```
class Demo {
        static int x = 10;
        void display() {
        System.out.println(x);
        }
}
```

Static Block

A static block executes only once, when the class is loaded.

```
class Demo {
        static {
        System.out.println("Static Block");
        }

        public static void main(String[] args) {
        System.out.println("Main Method");
        }
}
```

Order of execution: 1) Static block, 2) main(), 3) Constructor (when an object is created).

static vs Instance Members

Static	Instance
Belongs to class	Belongs to object
One copy	One copy per object
Access using class name	Access using object
Loaded once	Created with each object

Instance Initialization Block (Non-static Block)

An instance initialization block (IIB) is a block of code inside a class without any method name and without the static keyword.

```
class Demo {
        {
        System.out.println("Instance Block");
        }
}
```

When does it execute? Every time an object is created — the instance block runs before the constructor for every object.

Why do we use it? To perform common initialization that every constructor should execute. Nowadays it is rarely used because constructors usually make code clearer.

Order: Static block (once for each class) → Instance block → Constructor.

Difference: Static Block vs Instance Block

Static Block	Instance Block
Runs once	Runs every time an object is created
Uses static {}	Uses {}
Executes when the class is loaded	Executes before the constructor
Used for class-level initialization	Used for object-level initialization

final Keyword

The final keyword is used to restrict modification. It can be used with:

- Variable
- Method
- Class

Think of it as: "Once finalized, it cannot be changed."

- Final Variable — a final variable can be assigned only once
- Final reference variable — final prevents changing the reference; example: final Student s = new Student(); is allowed
- Final Method → a final method cannot be overridden
- Final class → cannot be inherited

final vs finally vs finalize()

final	finally	finalize()
Keyword	Block	Method
Restricts modification	Executes cleanup code	Called by Garbage Collector (legacy)
Used with variable, method, class	Used in exception handling	Defined in the Object class

Packages

A package is a collection of related classes and interfaces. Think of it like a folder on your computer.

Instead of keeping every class in one place, we organize them into folders — each folder is a package.

Why Do We Use Packages?

- Organization — group similar classes together instead of hundreds of classes in one folder (this is exactly how Spring Boot projects are organized)
- Avoid Name Conflict — two developers can each create a class named Student.java; with packages, college.Student and school.Student can both exist
- Access Protection — packages work together with access modifiers to control visibility

Creating a Package

At the top of the Java file:

```
package student;

public class Student {
}
// Student now belongs to the package "student"
```

Importing a Package

```
import student.Student;
// or
import student.*;

Student s = new Student();
```

Types of Packages

1. Built-in Packages — provided by Java:

- java.util
- java.lang

- java.io
- java.sql
- java.time

2. User-defined Packages — created by us / the developer:

```
package employee;
```

package vs import

package: tells Java "this class belongs here." import: tells Java "I want to use this class."

Access Modifiers

Access modifiers decide who can access a class, variable, constructor, or method. There are 4 types:

- private — accessible only inside the same class; to access it from another class we use getter and setter methods
- default (no keyword) — accessible inside the package, not accessible outside the package
- protected — accessible in the same class, same package, and child classes in another package
- public — accessible everywhere

Exception Handling

What Is an Exception?

An exception is an unexpected event that occurs during program execution and interrupts the normal flow of the program.

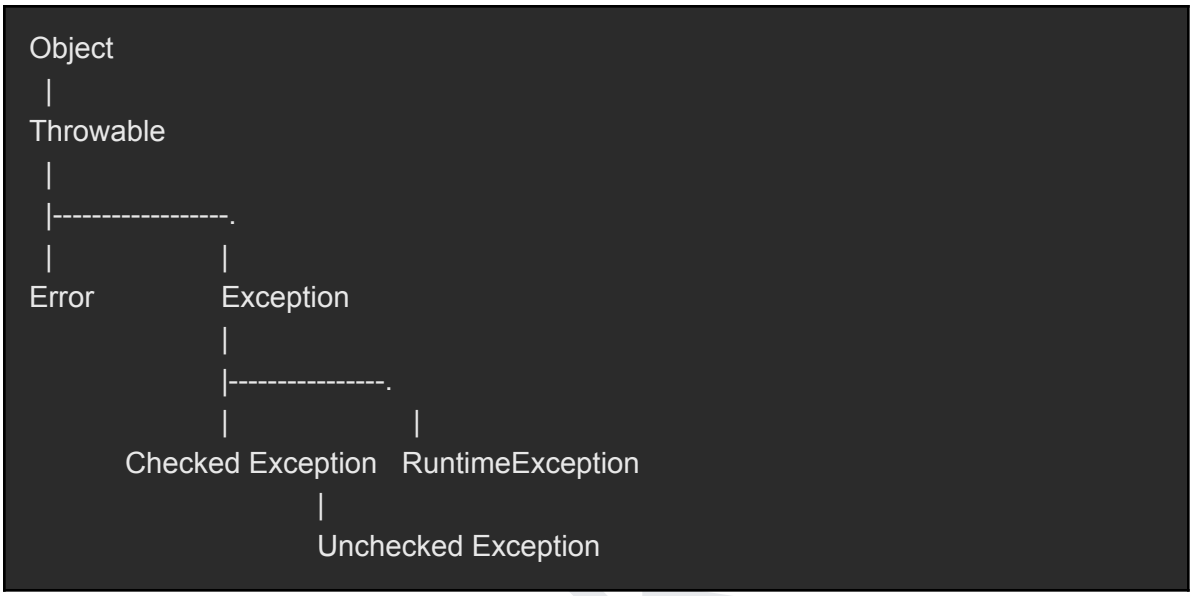
```
int a = 10;
int b = 0;
System.out.println(a / b);
// Exception in thread "main" java.lang.ArithmeticException
// The program crashes.
```

Why Do We Need Exception Handling?

Without exception handling: Program starts → exception occurs → program stops.

With exception handling: Program starts → exception occurs → exception handled → program continues.

Exception Hierarchy



Types of Exceptions

1. Checked Exception — checked at compile time. Example:

- IOException
- SQLException
- FileNotFoundException

The compiler forces you to handle them. Example:

```
FileReader file = new FileReader("abc.txt");
// If not handled: compile-time error
```

2. Unchecked Exception — occurs during runtime. Example:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException

The compiler doesn't force you to handle them.

Difference: Checked vs Unchecked Exception

Checked Exception	Unchecked Exception
Checked at compile time	Occurs at runtime
Must be handled	Optional to handle
Example: IOException	Example: ArithmeticException

Keywords Used

There are 5 keywords:

- try
- catch
- finally
- throw
- throws

try-catch

```
public class Demo {
        public static void main(String[] args) {
        try {
        int a = 10;
        int b = 0;
        System.out.println(a / b);
        } catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero");
        }
        System.out.println("Program continues");
        }
}
// Output:
// Cannot divide by zero
// Program continues
```

Multiple Catch Blocks

```
try {

} catch (ArithmeticException e) {

} catch (NullPointerException e) {

} catch (Exception e) {

}
```

Java checks from top to bottom; the first matching catch block executes.

Why Is Exception Written Last?

Writing catch(Exception e) before catch(ArithmeticException e) causes a compile-time error, because Exception is the parent of ArithmeticException — the parent catch block would already catch everything.

finally Block

The finally block executes whether an exception occurs or not.

```
try {
        int a = 10;
} catch (Exception e) {
        System.out.println(e);
} finally {
        System.out.println("Finally executed");
}
// Output: Finally executed
```

Why do we use finally? Mainly for cleanup — closing DB connections, closing files, closing sockets.

throw Keyword

Used to manually throw an exception.

```
int age = 15;
if (age < 18) {
        throw new ArithmeticException("Not Eligible");
}
// Output: Exception in thread "main" ArithmeticException: Not eligible
```

throws Keyword

Used in the method declaration to indicate that a method may throw an exception.

```
void readFile() throws IOException {
}
```

It tells the caller: "This method may throw an IOException; handle it or declare it further."

throw vs throws

throw	throws
Used inside a method	Used in method declaration
Throws one exception object	Declares one or more exception types
throw new ArithmeticException()	throws IOException

Custom Exception

We can create our own exception class.

```
class InvalidAgeException extends Exception {
        InvalidAgeException(String message) {
        super(message);
        }
}

// Use it:
if (age < 18) {
        throw new InvalidAgeException("Age must be 18 or above");
}
```

Custom exceptions make programs easier to understand because the exception's name describes the problem.

Common Runtime Exceptions

Exception	Cause
ArithmeticException	Divide by zero
NullPointerException	Using a null reference
ArrayIndexOutOfBoundsException	Invalid array index
NumberFormatException	Invalid string-to-number conversion
ClassCastException	Invalid object casting

Collections Framework

Suppose we want to store marks. Without collections:

```
int[] marks = {80, 90, 86};   // works fine
```

But arrays have problems.

Problems with Arrays

- Fixed in size — int[] arr = new int[5]; the size is fixed, we cannot increase it later
- Same data type — int[] arr = {10, "nitish"}; is an error, arrays can store only one data type
- Insertion & deletion — removing an element requires shifting remaining elements manually
- Searching — we often have to write loops to find elements

Why Collections?

Collections solve these problems. They provide:

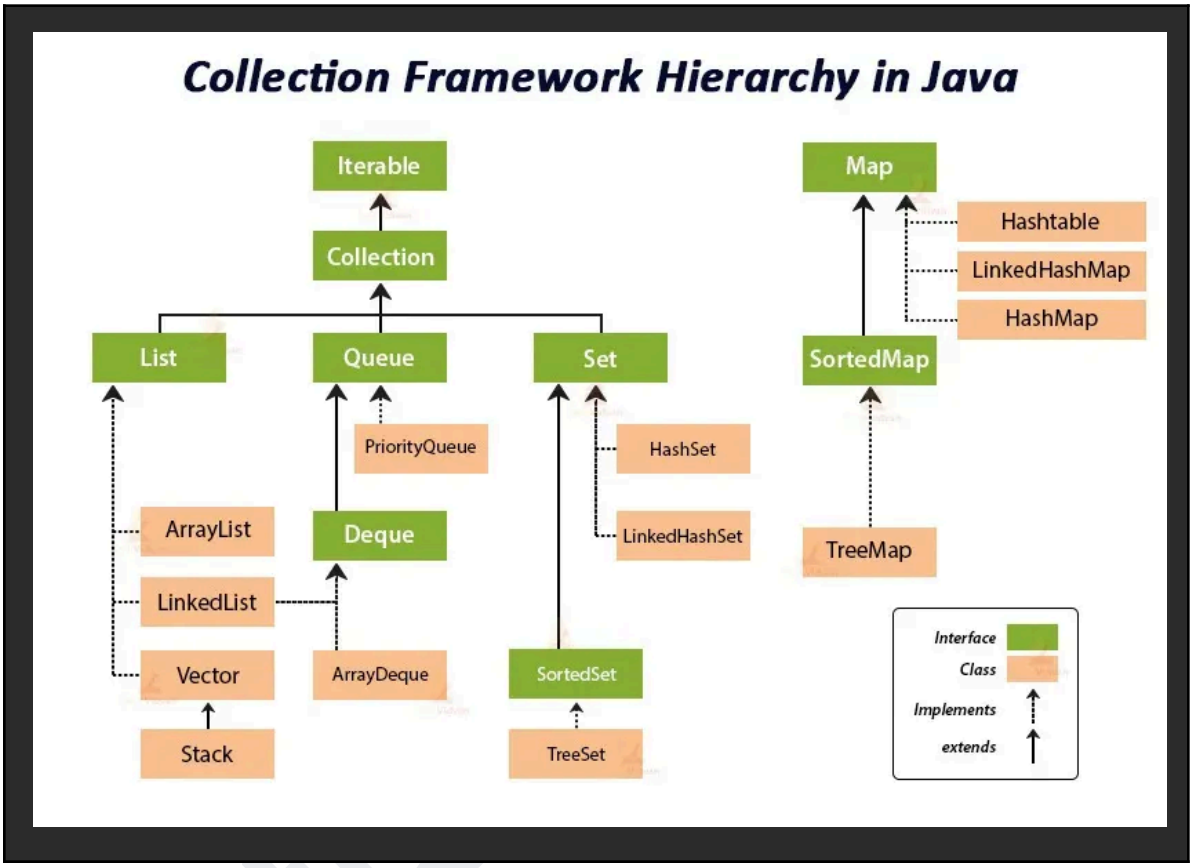
- Dynamic size
- Ready-made methods
- Fast searching
- Easier insertion / deletion

- Better data handling

What Is the Collection Framework?

The Collections Framework is a set of interfaces and classes used to store and manipulate groups of objects efficiently.

Collection Hierarchy



Main Interfaces

There are 4 main interfaces:

- List
- Set
- Queue
- Map

1. List

A list stores elements: in insertion order, allows duplicates, uses an index.

Classes that implement List:

- ArrayList
- LinkedList
- Vector
- Stack

ArrayList

Think of it as a dynamic array.

```
ArrayList<String> names = new ArrayList<>();
```

- Add element → names.add("Nitish");
- Print → System.out.println(names);
- Access element → names.get(indexValue);
- Update element → names.set(1, "Rohan");
- Remove by index → names.remove(index);
- Remove by object → names.remove("Rahul");
- Size → names.size();
- Check if element exists → names.contains("Nitish");
- Is empty → names.isEmpty();
- Clear list → names.clear();

Looping

```
// Using for loop
for (int i = 0; i < names.size(); i++) {
        System.out.println(names.get(i));
}

// Enhanced for loop
for (String name : names) {
        System.out.println(name);
}
```

Arrays vs ArrayList

Array	ArrayList
Fixed size	Dynamic size
Can store primitives directly	Stores objects (use wrapper classes like Integer)
Fewer methods	Many built-in methods
Faster for fixed-size data	More flexible

Why ArrayList<Integer>, Not ArrayList<int>?

Java Collections can store only objects, not primitive data types, so we use wrapper classes like Integer instead of int.

Can We Write ArrayList arr = new ArrayList()?

Yes, we can write this, but it can store anything. When we try to perform a calculation or operation on an element, it will throw an error, because Java does not know whether the object is an Integer, String, or Double — that is unsafe.

Java then introduced Generics — generics allow us to specify what type of data the collection can store.

LinkedList

A LinkedList is a class that implements the List interface. Like ArrayList, it:

- Maintains insertion order
- Allows duplicate elements
- Stores elements dynamically

Example output: [Nitish, Mehul, Rahul, Aman] — looks the same as ArrayList. The difference is not in the output, but in how the data is stored internally.

- ArrayList → data is stored in continuous memory (like an array)
- LinkedList → each element is called a Node, and each Node contains data and the address (reference) of the next node

Creating a LinkedList

```
LinkedList<String> names = new LinkedList<>();
```

- Add element: names.add("Nitish");

- Access element: names.get(1); — exactly like an ArrayList
- Update: names.set(index, value);
- Remove: names.remove("Rahul");
- Size: names.size();

Then Why Do We Even Need LinkedList?

Because some operations are much faster: inserting at the beginning — ArrayList is slow, LinkedList is faster. Random access — ArrayList is faster, LinkedList is slower.

Time Complexity

Operation	ArrayList	LinkedList
Get by index	O(1)	O(n)
Add at end	O(1) (amortized)	O(1)
Add at beginning	O(n)	O(1)
Remove at beginning	O(n)	O(1)

ArrayList → fast access. LinkedList → fast insertion / deletion (especially at the beginning or middle, when we already have the node).

ArrayList vs LinkedList

ArrayList	LinkedList
Uses Dynamic Array	Uses Doubly Linked List
Fast random access	Slow random access
Slow insertion in middle / beginning	Fast insertion / deletion
Less memory	More memory (stores references too)

Which One Should We Use?

Use ArrayList when: reading data frequently, searching by index, accessing elements randomly.

Use LinkedList when: frequent insertions, frequent deletions, queue-like operations.

Extra Methods in LinkedList

- Add first: names.addFirst("Nitish");

- Add last: names.addLast("Aman");
- Get first: names.getFirst();
- Get last: names.getLast();
- Remove first: names.removeFirst();
- Remove last: names.removeLast();

Vector

Vector is a class that implements the List interface. Like ArrayList, it:

- Maintains insertion order
- Allows duplicates
- Uses indexes
- Has dynamic size

```
Vector<String> names = new Vector<>();
```

Note: add(), get(), set(), remove() work the same as ArrayList.

Then Why Do We Need Vector?

The biggest difference is: Vector is synchronized.

ArrayList vs Vector

ArrayList	Vector
Not synchronized	Synchronized
Faster	Slower
Better for single-threaded programs	Better for multi-threaded programs
Introduced in Java 1.2	Legacy class (Java 1.0)

Stack

- A stack is a class that follows the LIFO principle
- LIFO = Last In, First Out
- The last element inserted is the first one removed

Stack Hierarchy: Object → Vector → Stack. Stack extends Vector, so it has all Vector methods like add(), remove(), size(), get() — but normally we use stack-specific methods.

```
Stack<String> books = new Stack<>();
```

- Add elements: books.push("Java");
- Remove top element: books.pop();
- Return the top element: books.peek();
- Check if the stack is empty: books.empty();
- Find an element: books.search("Java"); — returns its position from the top

2. Set

A Set is a collection that does not allow duplicate elements, does not use an index, and can store only unique values.

Collection Hierarchy: Collection → Set → HashSet, LinkedHashSet, TreeSet.

1. HashSet

- No duplicates
- No insertion order guarantee
- Allows one null value
- Fast insertion and searching

```
HashSet<String> names = new HashSet<>();
```

- Adding elements: names.add("Nitish");
- Remove element: names.remove("Nitish");
- Size: names.size();
- Check: names.contains();
- Check empty: names.isEmpty();
- Clear all: names.clear();

Traversing a HashSet

Since there is no index, names.get(i) is not possible — it causes a compile-time error. Instead, we use:

```
for (String name : names) {
        System.out.println(name);
```

```
}
```

A HashSet uses a technique called hashing.

Difference between List and Set

List	Set
Allows duplicates	No duplicates
Uses index	No index
Preserves insertion order	Depends on implementation
get(index) available	Not available

Important Methods

- fruits.addAll(otherSet) — combines two sets and stores unique elements
- set1.containsAll(set2) — returns true only if every element of set2 exists in set1
- fruits.retainAll(otherSet) — keeps only common elements
- set1.removeAll(set2) — removes all common elements

2. LinkedHashSet

LinkedHashSet is a class that implements the Set interface. It has all the properties of HashSet, plus one extra feature: it maintains insertion order.

- No duplicates
- Maintains insertion order
- No indexing
- Allows one null
- Uses hashing internally

Why Was LinkedHashSet Introduced?

Suppose we insert: Apple, Mango, Banana, Orange.

HashSet output: Orange, Apple, Banana, Mango — HashSet does not guarantee insertion order.

LinkedHashSet output: Apple, Mango, Banana, Orange — exactly the same order in which you inserted the elements.

Why does it maintain order? Internally, LinkedHashSet uses a hash table (for fast searching) plus a doubly linked list (to remember insertion order).

```
Set<String> names = new LinkedHashSet<>();
```

Common methods: add(), remove(), contains(), size(), clear(), isEmpty(). Traversing: use a for-each loop, since there is no indexing.

Difference between HashSet and LinkedHashSet

HashSet	LinkedHashSet
No duplicates	No duplicates
Doesn't maintain insertion order	Maintains insertion order
Fast	Slightly slower
Allows one null	Allows one null

3. TreeSet

A TreeSet is a class that implements the Set interface.

- No duplicate elements
- Automatically sorts elements
- No indexing
- Does not allow null
- Stores elements in ascending order by default

```
Set<Integer> num = new TreeSet<>();
```

Important methods are the same as HashSet: add(), remove(), contains(), size(), clear(), isEmpty().

Special Methods

- num.first() → returns the smallest element
- num.last() → returns the largest element
- num.higher() → returns the next greater element
- num.lower() → returns the next smaller element

- num.ceiling() → returns the equal value, or the next greater value
- num.floor() → returns the equal value, or the next smaller value

Difference between All Three Sets

Feature	HashSet	LinkedHashSet	TreeSet
Duplicates	No	No	No
Insertion Order	No	Yes	No
Sorted	No	No	Yes
Null Allowed	One	One	No
Search Speed	Fastest	Fast	Slower

Note: TreeSet uses a Red-Black tree data structure internally.

3. Queue

A Queue follows the FIFO principle (First Come, First Serve). The first element inserted is the first one removed.

Queue Hierarchy: Collection → Queue (Interface) → PriorityQueue, ArrayDeque.

Common Queue Methods

- add()
- remove() — throws an exception if the queue is empty
- peek() — returns the first element, or null if empty
- poll() — removes the first element, or returns null if empty
- element() — returns the first element, or throws an exception if empty

PriorityQueue

A PriorityQueue is a Queue that does not follow insertion order. Instead, it serves elements according to priority.

By default: Numbers → smallest first (ascending). Strings → alphabetical order. The smallest element has the highest priority.

Output might not appear fully sorted, because PriorityQueue does not guarantee sorted iteration order — it only guarantees that peek() or remove() will always return the highest-priority element.

ArrayDeque

ArrayDeque stands for Array Double Ended Queue. It allows insertion and deletion from both ends.

- addFirst()
- addLast()
- removeFirst()
- removeLast()
- getFirst()
- getLast()

It acts as both a stack and a queue.

4. Map

Map stores data as Key → Value. Example: 101 → Nitish. Every entry has two parts: a key and a value.

Collection Hierarchy: Collection → List, Set, Queue. Map is not inside Collection — it is a separate interface.

Q: Does Map extend Collection? A: No.

1. HashMap

The most commonly used implementation of Map.

- Key must be unique
- Values can be duplicate
- One null key allowed
- Multiple null values allowed
- No insertion order guarantee

Important Methods

```
Map<Integer, String> students = new HashMap<>();
students.put(101, "Nitish");          // adding data
students.get(101);                    // getting value
students.put(101, "Mehul");           // updating value (keys cannot duplicate)
students.remove(101);                 // removing an entry
students.size();                      // number of key-value pairs
students.containsKey(101);            // check if key exists
students.containsValue("Mehul");      // check if value exists
```

Difference between HashMap and HashSet

HashSet	HashMap
Stores only values	Stores key-value pairs
No duplicate values	No duplicate keys
add()	put()
contains()	containsKey()

Traversing Using entrySet()

This is the best and most commonly used method.

```
import java.util.HashMap;
import java.util.Map;

public class HashMapDemo {
        public static void main(String[] args) {
        Map<Integer, String> students = new HashMap<>();
        students.put(101, "Nitish");
        students.put(102, "Rahul");
        students.put(103, "Aman");

        for (Map.Entry<Integer, String> entry : students.entrySet()) {
        System.out.println(entry.getKey() + " -> " + entry.getValue());
        }
        }
}
// Output:
// 101 -> Nitish
// 102 -> Rahul
// 103 -> Aman
```

What is Map.Entry? A Map.Entry represents one key-value pair — the entire pair is one entry.

Traversing Using keySet()

If you only need the keys:

```
for (Integer key : students.keySet()) {
        System.out.println(key);
}
```

Getting the value from the key:

```
for (Integer key : students.keySet()) {
        System.out.println(key + " -> " + students.get(key));
}
```

Note: This works, but it is less efficient than entrySet(), because for every key Java has to call get() to find the value.

Traversing Using values()

If we only need the values, use values():

```
for (String value : students.values()) {
        System.out.println(value);
}
```

Comparison

Method	Gives Keys	Gives Values
entrySet()	Yes	Yes
keySet()	Yes	With get()
values()	No	Yes

Common Methods

- students.keySet(); → returns all keys
- students.values(); → returns all values
- students.entrySet(); → returns all key-value pairs

Q: What does students.keySet() return? A: Set<Integer> (a Set of keys). And entrySet() returns Set<Map.Entry<Integer, String>>.

LinkedHashMap

It is a class that implements the Map interface, storing data as key-value pairs just like HashMap. The only major difference is that LinkedHashMap maintains insertion order.

Internal working: LinkedHashMap uses a hash table (for fast lookup) plus a doubly linked list (to maintain insertion order). Traversal and other operations are the same as HashMap.

When should we use LinkedHashMap? For example, storing recent searches, browser history, or user activity logs.

HashMap vs LinkedHashMap

HashMap	LinkedHashMap
No insertion order guarantee	Maintains insertion order
Faster	Slightly slower
One null key	One null key
Multiple null values	Multiple null values

TreeMap

It combines Map + sorting. A TreeMap is a class that implements the Map interface and stores data as key → value, just like HashMap. The big difference: TreeMap automatically sorts the keys (the values are not sorted).

Methods to add, traverse, and remove elements are the same as HashMap.

Special Methods

- firstKey() → returns the smallest key
- lastKey() → returns the largest key
- higherKey(102) → returns the next greater key
- lowerKey(102) → returns the next smaller key
- ceilingKey(102) → returns the same key (if it exists), otherwise the next greater key; null if neither exists
- floorKey(102) → returns the same key (if it exists), otherwise the next smaller key; null if neither exists

Difference between HashMap, LinkedHashMap and TreeMap

Feature	HashMap	LinkedHashMap	TreeMap

Order	No guarantee	Insertion order	Sorted by key
Duplicate Keys	No	No	No
Duplicate Values	Yes	Yes	Yes
Null Key	One	One	No
Null Values	Yes	Yes	Yes
Speed	Fastest	Fast	Slower
Internal Structure	Hash Table	Hash Table + Linked List	Red-Black Tree

When Should We Use Them?

- HashMap — need fast access: student id → name
- LinkedHashMap — need fast access + insertion order
- TreeMap — need data stored sorted by key

HashTable

Hashtable is a class that implements the Map interface. It stores data as key → value, just like HashMap.

Why Was Hashtable Introduced?

Before Java Collections became mature, Hashtable was one of the first classes used for storing key-value pairs. Nowadays, HashMap is preferred in most applications.

```
Hashtable<Integer, String> students = new Hashtable<>();
// or: Map<Integer, String> students = new Hashtable<>();

students.put(101, "Nitish");   // adding data
students.get(102);             // getting data
students.put(102, "Rohit");    // updating (duplicate keys not allowed, value updated)
students.remove(103);          // removing data
```

Note: students.put(null, "Java") and students.put(101, null) both throw a NullPointerException. Hashtable allows neither null keys nor null values.

What Is Synchronization?

This is the main difference. Hashtable is synchronized — if two threads try to modify it at the same time, Java controls access so that only one thread modifies it at a time.

HashMap vs Hashtable

- HashMap — no synchronization, both threads may access it simultaneously, faster but not thread-safe
- Hashtable — synchronized, one thread waits while the other finishes, slower but thread-safe

Complete Map Comparison

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable
Order	No guarantee	Insertion order	Sorted by key	No guarantee
Duplicate Keys	No	No	No	No
Duplicate Values	Yes	Yes	Yes	Yes
Null Key	One	One	No	No
Null Values	Yes	Yes	Yes	No
Thread Safe	No	No	No	Yes
Speed	Fastest	Fast	Slower	Slower
Internal Structure	Hash Table	Hash Table + Linked List	Red-Black Tree	Hash Table

Comparable & Comparator

Problem Statement

Suppose we have an ArrayList<Integer>:

```
ArrayList<Integer> list = new ArrayList<>();
list.add(50);
list.add(20);
list.add(40);
list.add(10);

Collections.sort(list);
System.out.println(list);
// Output: [10, 20, 40, 50] — that is easy
```

But what if we have our own class?

```
class Student {
        int id;
        String name;
        double marks;
}
```

Now suppose we have: 101 Nitish 85, 102 Rahul 92, 103 Aman 75. Java doesn't know: should it sort by ID? By name? By marks? This is exactly why Comparable and Comparator exist.

Comparable

Comparable is an interface used to define the default sorting order of objects. Package: java.lang.

Comparable has only one method: compareTo()

```
public int compareTo(T obj)
```

How compareTo() Works

There are 3 possible return values:

- Negative number → current object comes first
- Zero → both objects are equal
- Positive number → current object comes after

```
10.compareTo(20)   // returns negative — 10 comes before 20
20.compareTo(20)   // returns 0 — equal
30.compareTo(20)   // returns positive — 30 comes after 20
```

Sorting Students by ID

```
import java.util.*;

class Student implements Comparable<Student> {
        int id;
        String name;

        Student(int id, String name) {
        this.id = id;
        this.name = name;
        }

        @Override
        public int compareTo(Student s) {
        return this.id - s.id;
        }

        @Override
        public String toString() {
        return id + " " + name;
        }
}

ArrayList<Student> list = new ArrayList<>();
list.add(new Student(103, "Aman"));
list.add(new Student(101, "Nitish"));
list.add(new Student(102, "Rahul"));

Collections.sort(list);
System.out.println(list);
// Output:
// 101 Nitish
// 102 Rahul
// 103 Aman
```

Automatically sorted by ID, because we implemented Comparable<Student> — Java now knows the default sorting rule.

Sorting in Descending Order

Instead of return this.id - s.id; write return s.id - this.id;

Sorting by name gives alphabetical order.

Sorting by marks: return Double.compare(this.marks, s.marks); — this is better than subtraction for floating-point numbers.

Comparator

Comparator is another interface. Package: java.util. Method: compare(obj1, obj2).

Why Comparator?

Suppose a student has Id, Name, and Marks. Sometimes we want to sort by Id, sometimes by Name, sometimes by Marks. Comparable cannot do that easily, but a Comparator can.

Example: Sort by Name

```
import java.util.Comparator;

class NameComparator implements Comparator<Student> {
        @Override
        public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name);
        }
}

// Now:
Collections.sort(list, new NameComparator());
// Output:
// Nitish
// Rahul
```

Sort by Marks

```
class MarksComparator implements Comparator<Student> {
        @Override
        public int compare(Student s1, Student s2) {
        return Double.compare(s1.marks, s2.marks);
        }
}
```

Comparable vs Comparator

Comparable	Comparator
Package: java.lang	Package: java.util
Method: compareTo()	Method: compare()
One default sorting	Multiple sorting logics
Modifies original class	Separate class
Natural ordering	Custom ordering

Note: Comparable is used when the class has one natural / default sorting order (e.g. by Id). Comparator is used when we need multiple sorting criteria without changing the original class.

Iterator

An Iterator is an interface used to traverse (iterate through) a collection one element at a time. Instead of using a for loop with an index, or a for-each loop, we can use an Iterator.

Why Do We Need an Iterator?

```
ArrayList<String> list = new ArrayList<>();
list.add("Nitish");
list.add("Rahul");
list.add("Aman");
```

We want to visit every element — an Iterator does this one by one.

```
Iterator<String> itr = list.iterator();
```

Note: list.iterator() returns an Iterator.

Important Methods

The Iterator has only three important methods:

- hasNext() → returns true if another element exists, otherwise false: itr.hasNext()
- next() → moves to the next element and returns it: itr.next()
- remove() → removes the current element safely: itr.remove()

Complete Example

```
import java.util.ArrayList;
import java.util.Iterator;

public class Demo {
        public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Nitish");
        list.add("Rahul");
        list.add("Aman");

        Iterator<String> itr = list.iterator();
        while (itr.hasNext()) {
        System.out.println(itr.next());
        }
        }
}
// Output:
// Nitish
// Rahul
// Aman
```

Note: With a for-each loop, it is not possible to safely remove an element — it will throw a ConcurrentModificationException. Use Iterator.remove() instead.

Generics

Generics allow us to specify the type of data that a class, interface, or method can work with, providing type safety at compile time.

Before Generics

```
ArrayList list = new ArrayList();
list.add("Java");
list.add(100);
list.add(true);
// Allowed — ArrayList stores everything as an Object
```

```
String s = (String) list.get(0);   // Works
String s = (String) list.get(1);   // list.get(1) is actually an Integer
// Runtime error: ClassCastException
```

Problems without generics: no type safety, need for type casting, errors occur at runtime.

After Generics

```
ArrayList<String> list = new ArrayList<>();
list.add("Java");
list.add("Spring");
// Now Java knows: only Strings are allowed.
```

The Object Class

Every class in Java implicitly extends Object if it doesn't extend another class.

```
class Student {
}

// Internally Java treats it as:
class Student extends Object {
}
```

Why Is the Object Class Important?

Because every Java object gets methods like:

- toString()

- equals()
- hashCode()
- getClass()
- clone()
- wait()
- notify()

toString()

```
class Student {
        int id = 101;
        String name = "Nitish";
}

// Main:
Student s = new Student();
System.out.println(s);
// Output: Student@5acf9800
```

Internally this calls System.out.println(s.toString()); the default toString() in Object returns ClassName@HexadecimalHashCode.

Overriding toString()

```
class Student {
        int id;
        String name;

        Student(int id, String name) {
        this.id = id;
        this.name = name;
        }

        @Override
        public String toString() {
        return "Student{id=" + id + ", name='" + name + "'}";
        }
}
```

```
// Output: Student{id=101, name='Nitish'} — much more readable
```

equals()

```
String s1 = new String("Java");
String s2 = new String("Java");

System.out.println(s1 == s2);            // false — compares references
System.out.println(s1.equals(s2));   // true — compares content
```

```
class Student {
        int id;
        String name;
        Student(int id, String name) {
        this.id = id;
        this.name = name;
        }
}

// Main:
Student s1 = new Student(101, "Nitish");
Student s2 = new Student(101, "Nitish");
System.out.println(s1.equals(s2));
// Output: false
```

Why? Because Student inherits the default equals() from Object, which behaves like == (compares references).

Overriding equals()

```
@Override
public boolean equals(Object obj) {
        if (this == obj)
        return true;
        if (obj == null || getClass() != obj.getClass())
        return false;
        Student other = (Student) obj;
        return id == other.id && name.equals(other.name);
}

// Output now: true — comparing the object's data instead of the memory address
```

hashCode()

Every object has a hash code.

```
Student s = new Student(101, "Nitish");
System.out.println(s.hashCode());
// Output: some integer value
```

Hash-based collections like HashMap, HashSet, and Hashtable use hash codes to decide where to store objects.

Why Override hashCode()?

Suppose s1.equals(s2) returns true — then s1.hashCode() and s2.hashCode() must also be equal. This is called the equals-hashCode contract.

Correct override:

```
import java.util.Objects;

@Override
public int hashCode() {
        return Objects.hash(id, name);
}
```

Multithreading

What Is a Process?

A process is a program that is currently running. Examples: Chrome, VS Code, Spotify, IntelliJ IDEA. When we open Chrome, chrome.exe starts running — that is called a process.

What Is a Thread?

A thread is the smallest unit of execution inside a process. A process can have multiple threads.

Process vs Thread

Process	Thread
Independent program	Small part of a process
Has its own memory	Shares process memory
Heavyweight	Lightweight
Creation is slower	Creation is faster
Communication is slow	Communication is fast

What Is Multithreading?

Multithreading is the process of executing multiple threads concurrently within the same process. Simple meaning: more than one task runs at the same time.

Thread Life Cycle

Every thread goes through different states:

```
New → Runnable → Running → Waiting / Blocked → Runnable → Running → Terminated
```

- New: thread object is created (Thread t = new Thread();) — the thread exists, but has not started
- Runnable: when we call t.start(); the thread becomes runnable — ready to run and waiting for the CPU
- Running: the CPU selects the thread and its run() method starts executing
- Waiting / Blocked: the thread pauses, e.g. Thread.sleep(5000); or waiting for a lock — during this time the thread is not using the CPU
- Runnable again: after waiting is over, the thread becomes ready again
- Terminated: when run() finishes, the thread ends and cannot be restarted

Main Thread

Every Java program has one thread by default.

```
public class Demo {
        public static void main(String[] args) {
      System.out.println("Hello");
        }
}
```

Even if we don't create any threads, Java creates one automatically — that thread is called the main thread.

How to Get the Current Thread

```
public class Demo {
        public static void main(String[] args) {
        Thread t = Thread.currentThread();
        System.out.println(t);
        }
}
// Possible output: Thread[main,5,main]
// Meaning: Thread name → main, Priority → 5, Thread Group → main
```

Creating Threads

There are two ways to create a thread:

- Extend the Thread class
- Implement the Runnable interface

Method 1: Extending the Thread Class

The Thread class is present in java.lang.Thread. To create a thread: extend the Thread class, override the run() method, and call the start() method.

```
class MyThread extends Thread {
        @Override
        public void run() {
        System.out.println("Child Thread is running...");
        }
}

public class Demo {
        public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
        System.out.println("Main Thread");
        }
}
// Possible output:
// Child Thread is running...
// Main Thread
// (order can vary)
```

Why can the output change? Because thread scheduling is controlled by the operating system, not by Java. After calling start(), Java only requests the OS to run the new thread; the OS decides which thread gets the CPU first. This is called thread scheduling.

Why Override run()?

Because all the work of the thread is written inside run(). Whenever the thread starts, this method executes.

Method 2: Implementing Runnable

This is the recommended approach. Runnable is an interface with only one method: public void run();

```
class MyTask implements Runnable {
        @Override
        public void run() {
        System.out.println("Runnable Thread");
        }
}

public class Demo {
        public static void main(String[] args) {
        MyTask task = new MyTask();
        Thread t = new Thread(task);
        t.start();
        }
}
// Output: Runnable Thread
```

Why do we create a Thread object here? task is not a thread — it is only an object that contains the work. The actual thread is created with new Thread(task), and then t.start() starts that thread.

Thread Class vs Runnable

Thread	Runnable
Class	Interface
Uses inheritance	Uses interface implementation
Cannot extend another class	Can still extend another class
Less flexible	More flexible
Rarely preferred	Preferred in real projects

Why is Runnable preferred? Suppose class Employee extends Person, and we also need a thread. We cannot do class Employee extends Person, Thread — Java doesn't support multiple inheritance of classes. But we can do class Employee extends Person implements Runnable.

Difference between start() and run()

start()	run()

Creates a new thread	Normal method call
Executes concurrently	Executes in the current thread
Calls run() internally	Does not create a new thread

Common Thread Methods

Method	Purpose
sleep()	Pause current thread
join()	Wait for another thread to finish
yield()	Suggest giving CPU to another thread
setName()	Set thread name
getName()	Get thread name
setPriority()	Set thread priority
getPriority()	Get thread priority
currentThread()	Get currently executing thread

Synchronization

It is a mechanism that allows only one thread at a time to access a shared resource, preventing inconsistent data.

What Is a Race Condition?

A race condition occurs when: multiple threads access the same data, at least one thread modifies the data, and the final result depends on the order in which threads execute. This can produce incorrect results.

Synchronized Methods

Java provides the synchronized keyword.

```
class Counter {
        private int count = 0;

        public synchronized void increment() {
        count++;
        }

        public int getCount() {
        return count;
        }
}
```

Only one thread can execute increment() on the same Counter object at a time. If another thread calls it, it waits.

Synchronized Block

Sometimes we don't want to synchronize the entire method.

```
public void increment() {
        System.out.println("Logging");
        synchronized (this) {
        count++;
        }
        System.out.println("Done");
}
```

Only the block synchronized(this) { count++; } is synchronized — the logging statements can still run without locking.

Why Use a Synchronized Block?

Imagine a method that reads a file, updates a database, and prints a message. Only the DB update needs protection — no need to lock the whole method. This improves performance.

What Is a Lock?

Whenever a thread enters a synchronized method or block, Java automatically gives it a lock (also called a monitor).

Thread-Safe

A class is thread-safe if it works correctly when multiple threads access it simultaneously.

- Thread-safe examples: Hashtable, Vector, StringBuffer
- Not thread-safe: HashMap, ArrayList, HashSet

Deadlock

Deadlock is a situation where two or more threads wait for each other forever, and none of them can proceed. It occurs when multiple locks are used, or threads acquire them in different orders.

How to Prevent Deadlock?

- Always acquire locks in the same order
- Keep synchronized blocks as short as possible
- Avoid unnecessary nested synchronization

Difference between Deadlock and Starvation

Deadlock	Starvation
Threads wait for each other forever	A thread waits because other threads keep getting the CPU / resources
Usually involves locks	Usually involves scheduling or resource allocation
No thread makes progress	Other threads continue; one thread may never get a chance

Inter-Thread Communication

Why do we need it? Suppose two threads are working together — one thread produces data, another thread consumes data.

Question: what if the consumer arrives first and there is no data? Should it keep checking every second?

```
while (food == null) {
}
// This is called busy waiting — a waste of CPU.
```

Instead, Java provides:

- wait() — tells the current thread to release the lock and wait until another thread wakes it up

- notify() — wakes up one waiting thread; Java does not guarantee which thread wakes
- notifyAll() — wakes up all waiting threads

Note: wait() and notify() can only be called inside a synchronized method or block.

wait() vs sleep()

wait()	sleep()
Defined in the Object class	Defined in the Thread class
Releases the lock	Does NOT release the lock
Used for thread communication	Used to pause execution
Must be inside a synchronized block / method	Can be called anywhere (with proper exception handling)

Java 8 Features

Java 8 introduced a set of features that made Java more concise and functional. The main ones are:

- Functional Interface
- Lambda Expression
- Method Reference
- Predicate
- Function
- Consumer
- Supplier
- Stream API
- Optional
- Date & Time API

Functional Interface

A functional interface is an interface that contains exactly one abstract method.

```
interface Animal {
        void sound();
```

```
}
```

Default methods and static methods can also be present, but a functional interface must always have exactly one abstract method.

@FunctionalInterface

```
@FunctionalInterface
interface Animal {
        void sound();
}
```

Why use this annotation? It tells Java that this interface must always remain a functional interface — the compiler raises an error if a second abstract method is added.

Why were functional interfaces introduced? Because of lambda expressions — without a functional interface, lambda expressions would not work.

Lambda Expression

A lambda expression is a concise way of implementing the single abstract method of a functional interface, without creating a separate implementation class.

Step 1: The Normal Way

```
@FunctionalInterface
interface Greeting {
        void sayHello();
}

class Welcome implements Greeting {
        @Override
        public void sayHello() {
      System.out.println("Hello");
        }
}

// Main:
Greeting g = new Welcome();
g.sayHello();
```

Step 2: Anonymous Class

Why create a separate class? Instead:

```
Greeting g = new Greeting() {
        @Override
        public void sayHello() {
      System.out.println("Hello");
        }
};
```

No Welcome class is needed. This is called an anonymous class.

Step 3: Lambda Expression

```
Greeting g = () -> {
        System.out.println("Hello");
};
```

Done — no class, no implements, no @Override, no anonymous class. Much clearer.

() -> { } is called a lambda expression:

- () → parameters
- -> → the lambda operator
- { } → the method body

Shortcut for a Single Statement

If the lambda expression has one statement:

```
(a, b) -> {
        return a + b;
}

// Can be written as:
(a, b) -> a + b;
```

Anonymous Class vs Lambda

Anonymous Class	Lambda Expression
More boilerplate code	Very concise
Creates a separate .class file	No extra class file generated
Can implement interfaces with multiple methods	Only works with functional interfaces (single abstract method)
this refers to the anonymous class instance	this refers to the enclosing class instance

Method Reference (::)

Why was method reference introduced? Suppose we have this lambda expression:

```
(a, b) -> a + b
```

This is already short. But what if our lambda expression only calls an existing method?

```
name -> System.out.println(name);
```

Here, the lambda isn't doing any new work — it simply calls System.out.println(name). Java says: if our lambda only calls an existing method, we can replace it with a method reference.

Lambda vs Method Reference

```
// Lambda:
name -> System.out.println(name)

// Method reference:
System.out::println
```

Both do exactly the same thing — the second one is cleaner.

The :: operator means "use this existing method as the implementation." It is called the method reference operator.

Example 1: Static Method Reference

```
class Calculator {
        public static int square(int n) {
        return n * n;
        }
}

@FunctionalInterface
interface Square {
        int find(int n);
}

// Lambda
Square s = n -> Calculator.square(n);
System.out.println(s.find(5));

// Method reference
Square s = Calculator::square;
System.out.println(s.find(5));
```

Example 2: Instance Method Reference

```
class Message {
        public void display(String msg) {
        System.out.println(msg);
        }
}

@FunctionalInterface
interface Printer {
        void print(String msg);
}

// Lambda:
Message m = new Message();
Printer p = text -> m.display(text);
p.print("Hello");

// Method reference:
Message m = new Message();
```

```
Printer p = m::display;
p.print("Hello");
// Output: Hello
```

Example 3: Method Reference to System.out.println

```
List<String> names = Arrays.asList("Nitish", "Rahul", "Aman");

// Lambda
names.forEach(name -> System.out.println(name));

// Method reference
names.forEach(System.out::println);
// Both print: Nitish, Rahul, Aman
```

Types of Method References

There are 4 types of method references:

- Static method
- Instance method (of a particular object)
- Instance method (of an arbitrary object of a type)
- Constructor reference

Predicate

Predicate is a functional interface that takes one input and returns a boolean value (true or false).

Package: import java.util.function.Predicate;

```
boolean test(T t);
```

Note: test() is the only abstract method — that's why Predicate is a functional interface.

Why Was Predicate Introduced?

Suppose we want to check: is a number even, is a person eligible to vote, is a salary greater than 50000, is a string empty? Normally we would write a separate method every time:

```
public boolean isEven(int n) {
        return n % 2 == 0;
}
```

Java 8 says: instead of creating separate methods, use a Predicate.

```
Predicate<DataType> variable = (parameter) -> condition;

// Example:
Predicate<Integer> even = n -> n % 2 == 0;
```

```
import java.util.function.Predicate;

public class Main {
        public static void main(String[] args) {
        Predicate<Integer> even = n -> n % 2 == 0;

        System.out.println(even.test(10));
        System.out.println(even.test(7));
        }
}
// Output:
// true
// false
```

Predicate Methods

and() — performs a logical AND operation:

```
Predicate<Integer> greater = age -> age > 18;
Predicate<Integer> smaller = age -> age < 60;

Predicate<Integer> eligible = greater.and(smaller);
System.out.println(eligible.test(25));
```

or() — performs a logical OR operation.

negate() — reverses the result:

```
Predicate<Integer> even = n -> n % 2 == 0;
Predicate<Integer> odd = even.negate();

odd.test(5);   // returns true
```

Function

Function is a functional interface that accepts one input and returns one output.

Package: import java.util.function.Function;

```
R apply(T t);
```

T → input type, R → return type. This is the biggest difference from Predicate, which always returns a boolean.

Why Was Function Introduced?

Suppose we want to convert lowercase to uppercase, find the square of a number, convert an Integer to a String, or find the length of a String. Normally we would write a separate method each time. Java 8 says: instead, use a Function.

```
Function<InputType, ReturnType> variable = parameter -> result;

// Example:
Function<Integer, Integer> square = n -> n * n;
```

Example: Square of a Number

```
import java.util.function.Function;

public class Main {
        public static void main(String[] args) {
        Function<Integer, Integer> square = n -> n * n;

        System.out.println(square.apply(5));
        System.out.println(square.apply(10));
        }
}
// Output:
// 25
// 100
```

Function Methods

- apply() → executes the function
- andThen() → executes Function A first, then Function B
- compose() → the opposite of andThen() — executes B first, then A
- identity() → simply returns the input unchanged

Predicate vs Function

Predicate	Function
Returns boolean	Returns any type (R)
Method: test()	Method: apply()
Used for conditions / filtering	Used for transformation

Consumer

Consumer is a functional interface that accepts one input and returns no result (void).

Package: import java.util.function.Consumer;

```
void accept(T t);
```

Why Was Consumer Introduced?

Suppose we only want to print a name, save data, display a message, or log information.

```
Consumer<DataType> variable = parameter -> {
        // code
};

// Example:
Consumer<String> print = name -> System.out.println(name);
```

```
import java.util.function.Consumer;

public class Main {
        public static void main(String[] args) {
        Consumer<String> print = name -> System.out.println(name);
        print.accept("Nitish");
        }
}
// Output: Nitish
```

Consumer Methods

Consumer has 2 important methods:

- accept() → executes the consumer
- andThen() → executes Consumer A, then Consumer B

Predicate vs Function vs Consumer

Predicate	Function	Consumer
test(T t) → boolean	apply(T t) → R	accept(T t) → void
Used for filtering / conditions	Used for transformation	Used for performing an action
No side effect required	Returns a value	No return value

Supplier

Supplier is a functional interface that takes no input and returns a value.

Package: import java.util.function.Supplier;

```
T get();
```

Note: No input, only a return value.

Why Was Supplier Introduced?

Suppose we want to generate the current date, a random number, an OTP, or a default value.

```
Supplier<DataType> variable = () -> value;

// Example:
Supplier<String> name = () -> "Nitish";
```

```
import java.util.function.Supplier;

public class Main {
        public static void main(String[] args) {
        Supplier<String> name = () -> "Nitish";
        System.out.println(name.get());
        }
}
// Output: Nitish
```

get() → returns the supplied value.

```
Supplier<Integer> otp = () -> (int) (1000 + Math.random() * 9000);
System.out.println(otp.get());
```

Predicate vs Function vs Consumer vs Supplier

Interface	Input	Output	Abstract Method
Predicate	One	boolean	test()
Function	One	A result (R)	apply()

Consumer	One	None (void)	accept()
Supplier	None	A result (T)	get()

Stream API

What Is a Stream?

A stream is a sequence of elements that supports functional-style operations to process data without modifying the original data.

Suppose we have an ArrayList:

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40);
```

We want to: find even numbers, double every number, sort them, remove duplicates, find the maximum, count elements. Before Java 8 we used loops, but Java 8 says instead of writing loops every time, use streams.

```
numbers.stream()
        .filter(n -> n % 2 == 0)
        .forEach(System.out::println);
// Same result, much cleaner.
```

Why Was Stream API Introduced?

Before Java 8, if we wanted to filter, sort, or transform data, we wrote many loops — a lot of code. Java 8 introduced streams to:

- Reduce code
- Improve readability
- Support functional programming
- Enable parallel processing

Stream vs Collection

Collection	Stream
Stores data	Processes data

Can be iterated multiple times	Can be consumed only once
Eager (data is stored)	Lazy (intermediate operations run only when a terminal operation is called)
Modifies underlying data structure	Does not modify the original data

Stream Pipeline

Source → Intermediate Operation(s) → Terminal Operation

- Source — where the data comes from. Example: numbers.stream() — numbers is the source
- Intermediate Operations — transform the stream. Examples: filter(), map(), sorted(), distinct(), limit(), skip(). Intermediate operations are lazy.
- Terminal Operation — produces the final result. Examples: collect(), count(), forEach(), reduce(), findFirst(). Without a terminal operation, nothing happens.

```
numbers.stream()
        .filter(n -> n > 20)
        .map(n -> n * 2)
        .forEach(System.out::println);
```

Note: A stream can be used only once, because streams are consumed after one terminal operation. To use it again, we have to create a new stream.

How to Create a Stream

```
// From a List:
List<String> names = Arrays.asList("A", "B", "C");
Stream<String> stream = names.stream();

// From an Array:
String[] arr = {"Java", "SQL"};
Stream<String> stream = Arrays.stream(arr);

// Using Stream.of():
Stream<Integer> stream = Stream.of(10, 20, 30);
```

parallelStream()

Collections also provide parallelStream().

```
List<Integer> nums = Arrays.asList(10, 20, 30, 40);

nums.parallelStream()
        .forEach(System.out::println);
// Output order may vary, e.g. 30 10 40 20 —
// because multiple threads process the data simultaneously.
```

Intermediate Operations

filter()

It is an intermediate stream operation that selects elements based on a condition.

Why does filter() use a Predicate? A Predicate always returns true or false:

```
Predicate<Integer> even = n -> n % 2 == 0;
```

Now look at filter():

```
.filter(n -> n % 2 == 0)
```

This is exactly a Predicate. If the predicate returns true → keep the element; false → discard the element. That's why filter() accepts a Predicate.

map()

map() is an intermediate operation that transforms each element of a stream into another element.

Note: filter() decides whether to keep an element. map() decides how to change an element.

Why was map() introduced? Suppose we have:

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40);
```

We want to double every number. Before Java 8:

```
List<Integer> result = new ArrayList<>();
for (int num : numbers) {
        result.add(num * 2);
}
System.out.println(result);
// Output: [20, 40, 60, 80]
```

Using streams:

```
numbers.stream()
        .map(n -> n * 2)
        .forEach(System.out::println);
// Output is the same.
```

Syntax: stream.map(function) — map() uses a Function, not a Predicate or Consumer. Nothing is removed; everything is transformed.

Difference between filter() and map()

filter()	map()
Selects elements	Transforms elements
Uses a Predicate	Uses a Function
May reduce the number of elements	Number of elements stays the same

sorted()

It is an intermediate operation used to sort the elements of a stream.

- stream.sorted() → ascending order
- stream.sorted(Collections.reverseOrder()) → descending order

distinct()

It removes duplicate elements from a stream.

Note: For primitive wrapper types like Integer and String, distinct() relies on their correctly implemented equals() and hashCode() methods.

limit()

Returns only the first n elements of the stream.

```
List<Integer> nums = Arrays.asList(10, 20, 30, 40, 50);

nums.stream()
        .limit(3)
        .forEach(System.out::println);
// Output: 10, 20, 30
```

skip()

Ignores the first n elements and processes the remaining elements.

```
List<Integer> nums = Arrays.asList(10, 20, 30, 40, 50);

nums.stream()
        .skip(2)
        .forEach(System.out::println);
// Output: 30 40 50
```

Note: filter(), map(), sorted(), distinct(), limit(), and skip() are all intermediate operations, so none of them execute until a terminal operation (such as forEach() or collect()) is called.

Terminal Operations

A terminal operation is the final operation in a stream pipeline that produces a result or side effect, and closes the stream. In simple words: it starts the execution of the stream and gives the final output.

forEach()

A terminal operation used to perform an action on each element of the stream. It does not return anything; it simply performs an action.

Syntax: stream.forEach(action); — the action is usually a Consumer, so forEach() accepts a Consumer.

```
List<String> names = Arrays.asList("Nitish", "Rahul", "Aman");

names.stream()
        .forEach(System.out::println);
```

```
// Output: Nitish, Rahul, Aman
```

collect()

Used to collect the processed stream into a Collection or another data structure.

Note: After using filter() or map(), we often want a new List — that's exactly what collect() does.

```
List<Integer> nums = Arrays.asList(10, 15, 20, 25, 30);

List<Integer> even = nums.stream()
        .filter(n -> n % 2 == 0)
        .collect(Collectors.toList());

System.out.println(even);
// Output: [10, 20, 30]
```

Instead of printing, the result is stored in another list — collect(Collectors.toList()) collects the stream into a List.

count()

Returns the number of elements in the stream. The return type is long, not int.

```
List<Integer> nums = Arrays.asList(10, 15, 20, 25, 30);

long count = nums.stream()
        .filter(n -> n % 2 == 0)
        .count();

System.out.println(count);
// Output: 3
```

Why does count() return long? Imagine processing millions or billions of records — an int may overflow.

min()

Returns the smallest element in the stream, according to a Comparator.

Syntax: stream.min(comparator) — unlike sorted(), min() needs a comparator, because Java must know how to compare two elements.

```
import java.util.Arrays;
import java.util.List;

public class Main {
        public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(40, 10, 30, 20, 50);

        Integer smallest = nums.stream()
        .min(Integer::compareTo)
        .get();

        System.out.println(smallest);
        }
}
// Output: 10
```

Why .get()? Because min() does not return an Integer directly — it returns Optional<Integer>.

max()

Returns the largest element.

```
Integer largest = nums.stream()
        .max(Integer::compareTo)
        .get();

System.out.println(largest);
// Output: 50
```

findFirst()

Returns the first element.

```
List<String> names = Arrays.asList("Nitish", "Rahul", "Aman");

String first = names.stream()
        .findFirst()
        .get();

System.out.println(first);
// Output: Nitish
```

findAny()

Returns any one element from the stream. In a sequential stream, this is usually the first element.

```
String value = names.stream()
        .findAny()
        .get();

System.out.println(value);
// Usually: Nitish
```

anyMatch()

Uses a Predicate and returns a boolean — true if at least one element matches.

```
List<Integer> nums = Arrays.asList(10, 20, 30, 40);

boolean result = nums.stream()
        .anyMatch(n -> n > 25);

System.out.println(result);
// Output: true
```

allMatch()

Returns true only if all elements satisfy the condition.

```
boolean result = nums.stream()
        .allMatch(n -> n > 5);

System.out.println(result);
// Output: true — all are greater than 5
```

noneMatch()

Returns true if no element satisfies the condition.

```
boolean result = nums.stream()
        .noneMatch(n -> n < 0);

System.out.println(result);
// Output: true — because no element is negative
```

reduce()

A terminal operation that combines all elements of a stream into a single result.

Syntax: stream.reduce(identity, accumulator)

```
.reduce(0, (a, b) -> a + b)
```

- identity → the starting value (sum starts from 0)
- accumulator → (a, b) -> a + b; combines two values — a is the current accumulated result, b is the next element from the stream

Example: Sum Using reduce()

```
import java.util.Arrays;
import java.util.List;

public class Main {
        public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(10, 20, 30, 40);

        int sum = nums.stream()
        .reduce(0, (a, b) -> a + b);

        System.out.println(sum);
        }
}
// Output: 100
```

reduce() without Identity

```
Optional<Integer> sum = nums.stream()
        .reduce((a, b) -> a + b);

System.out.println(sum.get());
```

Note: With an identity, reduce() returns the raw value (e.g. int). Without an identity, it returns Optional<Integer>, because the stream may be empty.

Optional

Why Was Optional Introduced?

Before Java 8, there was a common problem:

```
String name = null;
System.out.println(name.length());
// Exception in thread "main" java.lang.NullPointerException
```

NullPointerException is one of the most common Java exceptions. Java 8 introduced Optional to reduce the chances of a NullPointerException.

What Is Optional?

Optional is a container object that may or may not contain a value. Instead of a plain String, we have Optional<String>, which may or may not be empty.

Creating Optional Objects

There are three important methods:

1. Optional.of() — used when we are 100% sure the value is not null.

```
import java.util.Optional;

public class Main {
        public static void main(String[] args) {
        Optional<String> name = Optional.of("Nitish");
        System.out.println(name);
        }
}
// Output: Optional[Nitish]
```

Note: If we pass null as the value to Optional.of(), the output is a NullPointerException.

2. Optional.ofNullable() — accepts both null and non-null values.

```
Optional<String> name = Optional.ofNullable("Nitish");
System.out.println(name);
// Output: Optional[Nitish]

Optional<String> name = Optional.ofNullable(null);
System.out.println(name);
// Output: Optional.empty — no exception
```

3. Optional.empty() — creates an empty Optional.

```
Optional<String> name = Optional.empty();
System.out.println(name);
// Output: Optional.empty
```

Checking and Getting the Value

```
Optional<String> name = Optional.of("Nitish");
```

isPresent() → checks the value: name.isPresent();

Getting the value: name.get(); — if the Optional is empty, the output is NoSuchElementException, so we should never call get() blindly.

Safe way:

```
if (name.isPresent()) {
        System.out.println(name.get());
}
```

Better Way: orElse()

Java 8 introduced orElse().

```
Optional<String> name = Optional.empty();
System.out.println(name.orElse("Guest"));
// Output: Guest
```

orElseThrow()

```
Optional<String> name = Optional.empty();
System.out.println(name.orElseThrow());
// Output: NoSuchElementException
```

Optional and Streams

```
List<Integer> nums = Arrays.asList(10, 20, 30);

Integer min = nums.stream()
        .min(Integer::compareTo)
        .get();
```

Methods like min(), max(), findFirst(), and reduce() (without identity) all return an Optional, because the stream might be empty.

Date & Time API

The main classes in the java.time package are: LocalDate, LocalTime, LocalDateTime, Period, and Duration.

1. LocalDate

It represents only the date (year, month, day), without time or timezone.

Get Current Date

```
import java.time.LocalDate;

public class Main {
        public static void main(String[] args) {
        LocalDate today = LocalDate.now();
        System.out.println(today);
        }
}
// Output: 2026-07-21
```

LocalDate.now() returns today's date according to your system clock.

Create a Specific Date

```
LocalDate birthday = LocalDate.of(2003, 10, 25);
System.out.println(birthday);
```

Get Individual Values

- today.getYear()
- today.getMonth()
- today.getMonthValue()
- today.getDayOfMonth()
- today.getDayOfWeek()

Add / Subtract

```
LocalDate today = LocalDate.now();
today.plusDays(10);
// The original object does not change — LocalDate is immutable.
```

- Subtract days: today.minusDays(5)
- Add months: today.plusMonths(2)
- Add years: today.plusYears(1)
- Is leap year: date.isLeapYear()
- Compare dates: d1.isBefore(d2) or d1.isAfter(d2)

2. LocalTime

It represents only time (hour, minute, second, and nanosecond), without date or timezone.

```
LocalTime time = LocalTime.now();   // current time

LocalTime time = LocalTime.of(10, 30, 45);   // custom time (hh, mm, ss)
```

- Get hour: time.getHour()
- Get minute: time.getMinute()
- Get second: time.getSecond()
- Add hour: time.plusHours(2)
- Add minutes: time.plusMinutes(20)
- Subtract hour: time.minusHours(3)
- Subtract minute: time.minusMinutes(30)
- Compare time: t1.isBefore(t2) or t1.isAfter(t2)

3. LocalDateTime

It represents both date and time, without a timezone. Example: 2026-07-21T10:45:30.

```
LocalDateTime curr = LocalDateTime.now();

// Create custom date & time:
LocalDateTime meeting = LocalDateTime.of(2026, 7, 22, 10, 30);
```

Get Individual Values

```
LocalDateTime now = LocalDateTime.now();

System.out.println(now.getYear());
System.out.println(now.getMonth());
System.out.println(now.getDayOfMonth());
System.out.println(now.getHour());
System.out.println(now.getMinute());
System.out.println(now.getSecond());
```

Add and Subtract

```
now.plusDays(10);
now.plusMonths(2);
now.plusYears(1);
now.plusHours(5);
now.plusMinutes(30);

now.minusDays(2);
now.minusHours(1);
now.minusMinutes(20);
```

Compare Date-Time

```
LocalDateTime d1 = LocalDateTime.of(2026, 1, 1, 10, 0);
LocalDateTime d2 = LocalDateTime.of(2026, 1, 1, 12, 0);

System.out.println(d1.isBefore(d2));
System.out.println(d1.isAfter(d2));
```

Period

Period represents the difference between two dates, in terms of years, months, and days. It works with LocalDate.

```
import java.time.LocalDate;
import java.time.Period;

public class Main {
        public static void main(String[] args) {
        LocalDate birthDate = LocalDate.of(2003, 10, 15);
        LocalDate today = LocalDate.now();

        Period age = Period.between(birthDate, today);
        System.out.println(age);
        }
}
// Output: P22Y9M6D — meaning 22 years, 9 months, 6 days
```

Get individual values:

```
age.getYears();
age.getMonths();
age.getDays();
```

Duration

Duration represents the difference between two times or date-times.

Period → works with dates. Duration → works with time / date-time.

```
import java.time.Duration;
import java.time.LocalTime;

LocalTime start = LocalTime.of(9, 30);
LocalTime end = LocalTime.of(12, 45);

Duration duration = Duration.between(start, end);
System.out.println(duration);
// Output: PT3H15M — meaning 3 hours 15 minutes
```

Get hours and minutes:

```
duration.toHours();
duration.toMinutes();
```

Note: toMinutes() returns the total minutes, not the remainder.

DateTimeFormatter

By default, LocalDate.now() prints in the format 2026-07-21. Sometimes we want 21/07/2026 or 21-Jul-2026 — this is why we use DateTimeFormatter.

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

LocalDate today = LocalDate.now();

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String result = today.format(formatter);

System.out.println(result);
// Output: 21/07/2026
```

Common Patterns

Pattern	Meaning	Example
dd	Day of month	21
MM	Month (number)	07
MMM	Month (short name)	Jul
yyyy	Year (4 digits)	2026
HH	Hour (24-hour)	10
mm	Minute	45
ss	Second	30

Format LocalDateTime

```
LocalDateTime now = LocalDateTime.now();

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy
HH:mm:ss");
System.out.println(now.format(formatter));
// Output: 21-07-2026 10:45:30
```

Parse a String into a Date

```
String date = "21/07/2026";

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
LocalDate d = LocalDate.parse(date, formatter);

System.out.println(d);
// Output: 2026-07-21
```